IndicWrite — Hinglish Smart Writing Assistant

CS3235 Working with LLMs · CIE3 Lab · RV University
Student: Munnaluri Rishikesh
USN: 1RVU23CSE291
Individual mini-project · Option 4
Submission deadline: November 6, 2026, 10:00 AM IST
Experimental evidence date: October 29, 2026

Abstract

IndicWrite is an Indian-language writing application that converts Roman-script Hindi (Hinglish) to native Devanagari script. The initial end-to-end implementation uses the **genuine AI4Bharat IndicXlit pretrained neural model**, rather than hand-written mappings or simulated responses. It provides a Streamlit user interface with text validation, example inputs, protected English words, error handling, model attribution, and copyable/downloadable conversion results.

In executed experiments on a **40-case manually authored evaluation dataset**, the system achieved a **macro-average character error rate (CER) of 5.58%**, **macro-average word accuracy of 82.67%**, and **55% exact sentence match**. Results demonstrate practical Roman-Hindi conversion while highlighting failure cases, particularly ambiguous shortened spellings. Model-based grammatical rewriting with BharatGen Param-1 and English translation with IndicTrans2 remain proposed improvements and are **not** represented as implemented.

1. Problem definition and relevance

Hindi writers often use the Latin alphabet on mobile keyboards and in informal conversations. This Roman-Hindi text is convenient to type but may need conversion into Devanagari for academic, official, or accessible communication. Because there is no single standard spelling for informal Roman Hindi, transliteration is nontrivial: `main`, `mai`, `mein`, and abbreviations such as `ap` can be interpreted differently. English technical words and proper nouns introduce additional ambiguity.

This project aims to provide a usable Indian-language AI solution and evaluate it quantitatively rather than treating any visually plausible Hindi output as proof of correctness.

2. Objectives and scope

1. Convert Roman-Hindi words into native Hindi script using a genuine pretrained Indic language model.
2. Retain punctuation, whitespace, digits, acronyms, and existing Devanagari; allow explicit English-term preservation.

3. Provide a responsive interface with sample input, conversion feedback, output copying, download and clear control.
4. Validate at least 30–50 test inputs, measure accuracy, and discuss failures with recorded predictions.
5. Keep actual model inference, unit-test stubs, and future features clearly separated.
6. Document architecture, model attribution, limitations, and repeatable setup.

The faculty's complete Option 4 describes transliteration, sentence improvement and optional translation. This version **completes and validates the transliteration application**, but **does not yet implement grammatical rewriting**. Translation is explicitly optional and likewise unimplemented.

3. Model and dataset selection

Component	Selection	Evidence status
Transliteration	AI4Bharat IndicXlit via `ai4bharat-transliteration==1.1.3`	**Pretrained model successfully executed**
Frontend	Streamlit with Python 3.10	**Ran and interacted in Chromium**
Writing improvement	Candidate BharatGen Param-1-2.9B-Instruct	**Not installed, integrated, or executed**
English translation	Candidate AI4Bharat IndicTrans2	**Not installed, integrated, or executed**
Training/data attribution	AI4Bharat Aksharantar	Cited as an official model resource, **not** claimed as the source of our manually authored tests

IndicXlit is a multilingual transformer supporting 21 Indic languages. The application uses the Hindi (`hi`) path. The official repository is MIT-licensed. The package's legacy Fairseq checkpoint required special loading compatibility with modern PyTorch. The other candidate models require independent checks on licensing, access, runtime and task suitability before integration.

4. System architecture and implementation methodology

```
Roman-Hindi text in Streamlit
       ↓
Input limits (500 characters / 80 Roman-script words)
       ↓
Tokenizer separates Latin words from spaces/punctuation/numbers/Hindi
       ↓
Preserve explicitly selected English terms and short uppercase acronyms
       ↓
Official AI4Bharat XlitEngine("hi", beam_width=4, rescore=False)
       ↓
Real model's top-1 Hindi prediction for each Roman word
       ↓
Reassemble converted and preserved tokens
       ↓
Streamlit results, copying and TXT download
```

The backend lazily initializes and caches the neural model once per Python process; within each request, repeated words use a small prediction cache. Each candidate Latin word is converted using the official `translit_word` interface. The application itself does not contain a hardcoded Hindi output dictionary. Errors from loading or inference are shown rather than replaced with fake AI responses.

The project contains `app.py`, `src/indicwrite/core.py`, `src/indicwrite/engine.py`, an executed `scripts/smoke_indicxlit.py`, `evaluation/evaluate.py`, and automated tests.

4.1 Compatibility engineering

On the first Ubuntu-based smoke run the official model package and roughly 121 MB model checkpoint downloaded successfully. A newer PyTorch build then rejected the legacy Fairseq checkpoint because `argparse.Namespace` was not allowlisted by default for restricted loading. We added a **narrowly scoped allowlist of that standard-library class when loading the official AI4Bharat weights**. The follow-up real-model run then succeeded. The application never accepts arbitrary third-party checkpoint files from users.

5. Experimental design

The evaluation corpus consists of **40 manually created sentences** in eight categories, with **five examples per category**:

- Ordinary Hinglish.
- Spelling variation and abbreviated Roman forms.
- Hindi-English loanwords.
- Names and locations.
- Punctuation.
- Code mixing with English terms kept in Latin script.
- Numbers and mixed alphanumeric tokens.
- Ambiguous Roman spellings.

Each record in `evaluation/cases.csv` has a unique ID, category, source Roman text, reference Hindi text, and optional English words to preserve. The recorded model output for every example is in `evaluation/results/predictions.csv`.

5.1 Metrics and definitions

For a reference string `R` and prediction `P`, character error rate (CER) is the Levenshtein distance at Unicode code-point level divided by the number of code points in `R`. Word error rate (WER) uses whitespace-delimited words and divides word-level edit distance by the reference word count. **Word accuracy is defined here as max(0, 1 − WER)**. Exact sentence match is 1 if normalized prediction equals normalized reference, otherwise 0. Each aggregate is a simple arithmetic average across sentences, **not a globally micro-weighted corpus score**.

These metrics measure agreement with authored references. Equivalent or acceptable variant spellings can receive nonzero error; the small corpus cannot establish nationwide model quality or official Aksharantar benchmark scores. This evaluation is for transliteration only, and does not measure grammatical rewriting.

6. Measured results

The genuine IndicXlit model ran on the full 40-case corpus in GitHub Actions, producing predictions, metrics and a failure sample file. **Observed results:**

Metric	Value
Cases executed	40 / 40
Mean CER	**5.58%**
Mean WER	**17.33%**
Mean word accuracy	**82.67%**
Exact sentence match	**22 / 40 (55%)**

6.1 Performance by category

Category	Cases	Mean CER	Word accuracy	Exact match
Ordinary Hinglish	5	2.02%	92.00%	60%
Spelling variation	5	17.06%	49.33%	0%
Loanwords	5	1.60%	96.00%	80%
Names	5	5.31%	82.00%	40%
Punctuation	5	8.58%	73.33%	40%
Code mixing	5	0.00%	100.00%	100%
Numbers	5	7.00%	87.00%	60%
Ambiguous	5	3.11%	81.67%	60%

Interpretation: The model handles common Roman Hindi, loanwords, and explicitly protected English terms well on this small evaluation set. Spelling variation is by far the most difficult category. The perfect code-mixing score partially reflects human-supplied lists of English terms to preserve, not autonomous English/Hindi language identification.

7. Error analysis

Real predictions were checked against the reference sentences; these examples are copied from the executed `worst_cases.csv`:

Roman input	Reference	Real model output	CER
`ap kahan ho?`	आप कहाँ हो?	एपी कहन हो?	36.36%
`mai theek hu.`	मैं ठीक हूँ.	माई ठीक हू.	25.00%
`5 minute ruk jao!`	5 मिनट रुक जाओ!	5 मिन्यूट रुक जाओ!	20.00%
`bengaluru se delhi jaunga.`	बेंगलुरु से दिल्ली जाऊँगा.	बेंगलुरु से देल्ही जौंगा.	19.23%

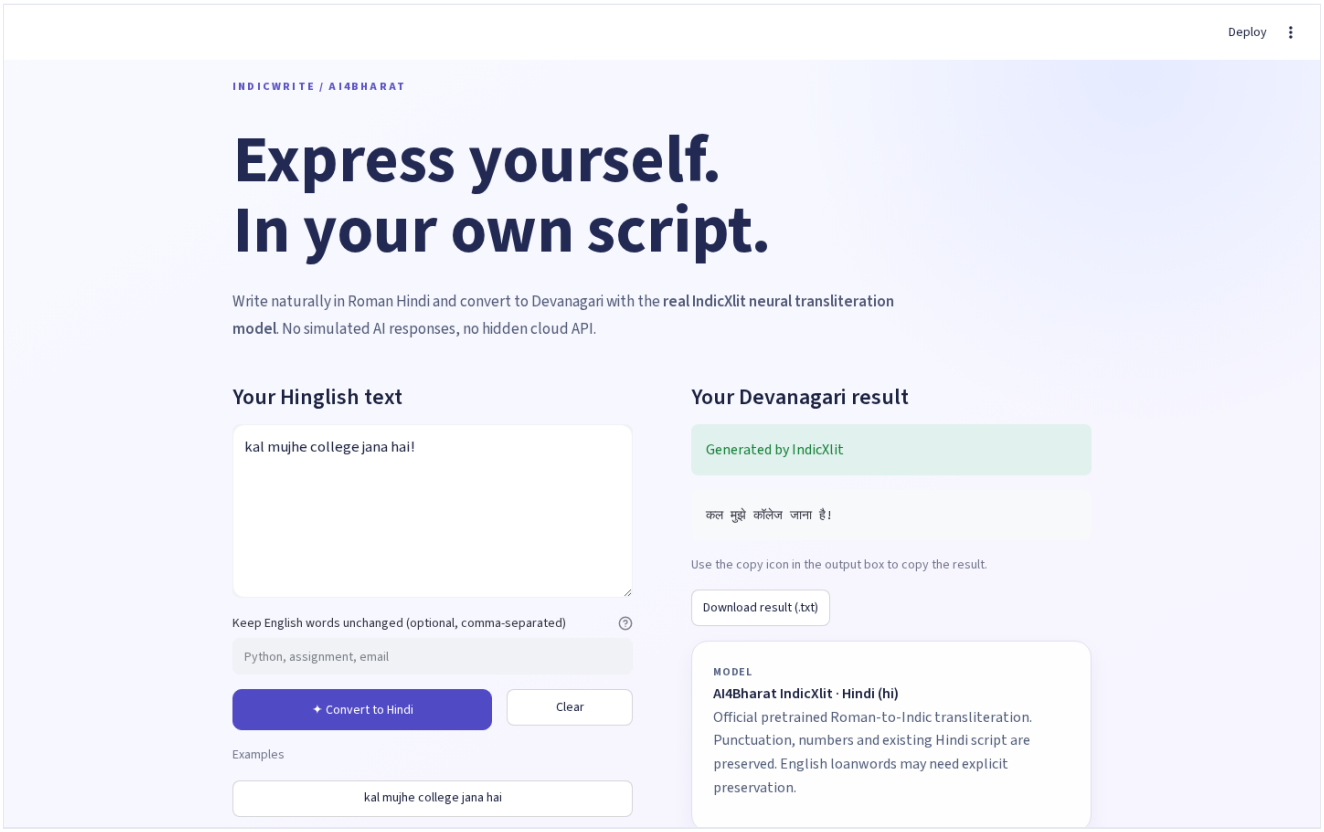
Roman input	Reference	Real model output	CER
`main aaunga; tum rukna.`	मैं आऊँगा; तुम रुकना.	मैन आऊंगा; टम रुकना.	19.05%

These are genuine model outputs, not examples synthesized to illustrate possible failures. Incorrect interpretations of short Roman words, Hindi vowels, and unfamiliar names are likely due in part to ambiguity without sentence-level context. These weaknesses motivate richer contextual reranking and constrained rewriting, but we do not claim they are fixed in the current MVP.

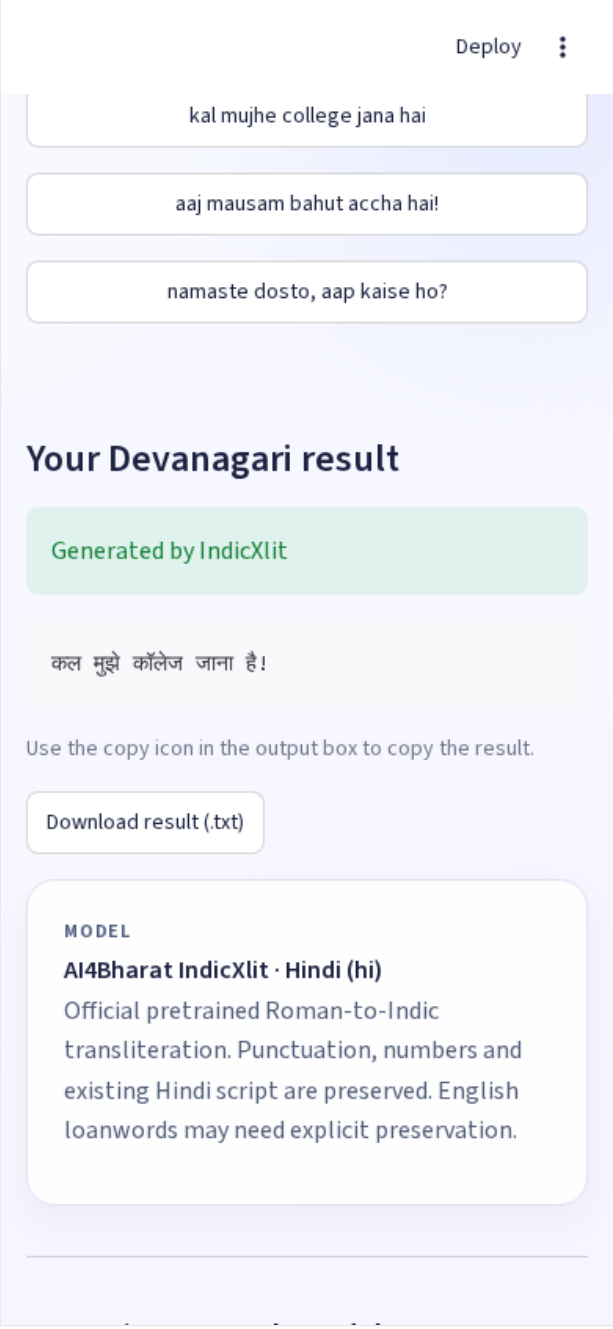
8. Executed testing and verification

- **Unit tests:** 14 tests passed on GitHub Actions. These test input handling, error propagation and metric calculations, and **use a fake engine** for deterministic text tests; their passing does not itself prove model inference.
- **Actual neural model:** Ubuntu 22.04, Python 3.10, downloaded official checkpoint. Smoke run `40870311669` printed `REAL_INDICXLIT_SMOKE_TEST_PASSED`.
- **Actual 40-case evaluation:** Run `40870722731` succeeded and committed authentic prediction/ metric files. The JSON and CSV are retained in GitHub.
- **Actual web UI/browser:** GitHub Actions Chromium Playwright run `40871267464` completed conversions and Clear actions on desktop at 1440px and mobile at 390px, with no detected page-level JavaScript errors or horizontal overflow. A follow-up run `40872355065` also passed and captured a mobile screenshot with the real result visible.

8.1 Actual desktop application screenshot



8.2 Actual mobile converted-result screenshot



Screenshot images are artifacts captured from the **running Streamlit application executing the real neural model**, not designer mockups. Both the screenshots and their browser QA JSON are committed in the repository.

9. Limitations and future work

The current model transliterates individual words; it **does not improve full-sentence grammar**, despite the broader smart-writing project name. No sentence-level semantic or contextual reranker is deployed. English terms are only preserved when manually specified or handled by a simple acronym rule. Some transliterations admit equally valid reference alternatives, so errors may overstate actual user dissatisfaction. The verification environment is Linux/Ubuntu; local native Windows installation may need WSL2 because of legacy Fairseq dependencies. No public deployment is asserted.

The next development phases would evaluate BharatGen Param-1 for grammatical rewriting (including meaning-preservation human evaluation), implement optional Hindi-to-English translation with IndicTrans2 if resources allow, add standard Aksharantar evaluation with correct splits and licensing, and consider context-aware candidate ranking, more nuanced mixed-script handling, and broader accessibility/device testing.

10. Conclusion

IndicWrite now provides an independently testable Indian-language AI transliteration application, with an accessible Streamlit interface, genuine AI4Bharat IndicXlit inference, forty measured cases, transparent failure analysis and real desktop/mobile screenshots. The reported performance and implementation scope are grounded in saved logs and results, while rewriting/translation remain unimplemented and explicitly documented.

References

1. AI4Bharat IndicXlit official repository and documentation.
2. Official AI4Bharat IndicXlit Python interface.
3. AI4Bharat Aksharantar dataset.
4. ai4bharat-transliteration on PyPI.
5. BharatGen Param-1-2.9B-Instruct model information.
6. AI4Bharat IndicTrans2 official repository.
7. PyTorch serialization behavior.
8. This project's recorded smoke test, evaluation run, and browser QA run.